

System Ostrzegania Pogodowego

Raport Końcowy z Projektu

Tomasz Pawłowski (198093), Antoni Sulkowski (197564)

Kurs: Autonomiczne systemy ekspertyzy i eksploracji danych

25 czerwca 2026

Spis treści

1	Wstęp i Cel Projektu	2
2	Architektura Systemu i Realizacja Kodowa	2
2.1	1. REST API Collector	2
2.2	2. Raw Storage	2
2.3	3. Validation / Cleaning	2
2.4	4. Transformation / Feature Engineering	3
2.5	5. Analytics / Expert Rules	3
2.6	6. Report / Dashboard / Presentation	3
3	Założenia Systemu, Reguły i Ograniczenia	4
3.1	Udokumentowane Założenia (Reguły Ekspertkie)	4
3.2	Ograniczenia Rozwiązania	4
4	Analiza Eksploracyjna Danych (Wyniki Analityczne)	4
4.1	Statystyki Opisowe i Metryki	4
4.2	Analiza Przebiegu Czasowego	4
4.3	Rozkład Prędkości Wiatru	5
4.4	Eksploracja Zależności (Korelacja)	6
4.5	Weryfikacja Systemu Powiadomień (SNS)	6
5	Podsumowanie	7

1 Wstęp i Cel Projektu

Niniejszy raport dokumentuje realizację zautomatyzowanego, bezserwerowego (serverless) pipeline-u danych w chmurze Amazon Web Services (AWS).

Głównym celem projektu było zaprojektowanie potoku, który cyklicznie pobiera surowe dane ze stacji meteorologicznej GDN_01 poprzez REST API, archiwizuje je, a następnie waliduje na podstawie eksperckich reguł pogodowych. System redukuje fałszywe alarmy i powiadamia użytkownika jedynie w sytuacjach przekroczenia zdefiniowanych progów bezpieczeństwa, gromadząc jednocześnie historię alertów w bazie NoSQL.

2 Architektura Systemu i Realizacja Kodowa

Architektura wdrożonego rozwiązania opiera się na usługach AWS: EventBridge, Lambda, S3, DynamoDB oraz SNS. Zgodnie z wytycznymi projektu, system realizuje zalecaną minimalną architekturę przetwarzania danych w sześciu krokach. Poniżej przedstawiono poszczególnych etapy w kodzie funkcji AWS Lambda.

2.1 1. REST API Collector

```
req = urllib.request.Request(API_URL)
req.add_header('Authorization', 'Bearer STUDENT_TOKEN_2026')

try:
    with urllib.request.urlopen(req) as response:
        raw_response = response.read().decode('utf-8')
        weather_data = json.loads(raw_response)
except Exception as e:
    print(f"B d API: {e}")
    return {"statusCode": 500, "body": "B d API"}
```

2.2 2. Raw Storage

Trwały zapis niezmiennych, surowych danych w formacie JSON bezpośrednio do magazynu Amazon S3.

```
file_name = f"raw_data/gdn_01_{timestamp_now}.json"
s3.put_object(
    Bucket=S3_RAW_BUCKET,
    Key=file_name,
    Body=json.dumps(weather_data)
)
```

2.3 3. Validation / Cleaning

Bezpieczne wyodrębnienie wartości numerycznych z wykorzystaniem metody `.get()`. Stanowi to formę walidacji zapobiegającą awariom skryptu w przypadku brakujących kluczy w odpowiedzi API.

```
wind_speed = weather_data.get('wind_speed', 0)
temperature = weather_data.get('temperature', 0)
rain_mm = weather_data.get('rain_mm', 0)
```

2.4 4. Transformation / Feature Engineering

Przekształcanie oraz rzutowanie zmiennych zmiennoprzecinkowych na format tekstowy, w celu zbudowania komunikatów oraz zachowania kompatybilności ze schematem bazy danych.

```
alert_message = " | ".join(alerts_triggered)
station = weather_data.get('station_id', 'GDN_01')

current_temp_str = str(temperature)
current_wind_str = str(wind_speed)
```

2.5 5. Analytics / Expert Rules

Zestaw reguł eksperckich.

```
alerts_triggered = []

if wind_speed > 15.0:
    alerts_triggered.append(f"Silny wiatr: {wind_speed} m/s")

if temperature < 2.0 and rain_mm > 0:
    alerts_triggered.append(f"Ryzyko go oledzi! Temp: {temperature}C,
        Opad: {rain_mm}mm")

if temperature > 35.0:
    alerts_triggered.append(f"Ekstremalny upa : {temperature}C")
```

2.6 6. Report / Dashboard / Presentation

Ostateczna prezentacja wyników i dystrybucja alertów realizowana dwutorowo: archiwizacja do raportów w bazie DynamoDB oraz natychmiastowe powiadomienie użytkownika poprzez usługę SNS.

```
table = dynamodb.Table(DYNAMODB_TABLE)
table.put_item(Item={
    'alert_id': str(uuid.uuid4()),
    'timestamp': timestamp_now,
    'station_id': station,
    'triggered_rules': alert_message,
    'current_temp': current_temp_str,
    'current_wind': current_wind_str
})

email_body = (
    f"OSTRZE ENIE POGODOWE DLA STACJI {station}\n"
    f"Czas: {timestamp_now}\n\n"
    f"Wykryte zagro enia:\n- {chr(10) + '- '.join(alerts_triggered)}\n"
)

sns.publish(
    TopicArn=SNS_TOPIC_ARN,
    Message=email_body,
    Subject=f"Alert Pogodowy - {station}"
)
```

3 Założenia Systemu, Reguły i Ograniczenia

3.1 Udokumentowane Założenia (Reguły Eksperckie)

- **Silny Wiatr (15.0 m/s):** Wiatr powyżej 15 m/s odpowiada 7. stopniowi w skali Beauforta, co stanowi realne zagrożenie.
- **Ryzyko Gołoledzi ($< 2.0^{\circ}\text{C}$ oraz opad > 0):** Zjawisko natychmiastowego zamarzania wody na podłożu - krytyczne dla ruchu drogowego.
- **Ekstremalny Upał ($> 35.0^{\circ}\text{C}$):** Próg ostrzegawczy ze względu na zagrożenie zdrowia.

3.2 Ograniczenia Rozwiązania

System opiera się na wyzwalaczu EventBridge działającym w interwale 15-minutowym. Oznacza to, że zjawiska gwałtowne i krótkotrwałe (np. 3-minutowy silny poryw wiatru podczas burzy) mogą zostać całkowicie pominięte, jeżeli wystąpią pomiędzy oknami odpytywania API.

4 Analiza Eksploracyjna Danych (Wyniki Analityczne)

W ramach testowania systemu wyekstrahowano próbkę 1422 ciągłych pomiarów z koszyka surowych danych (S3), używając bibliotek `pandas`, `matplotlib` oraz `seaborn`.

4.1 Statystyki Opisowe i Metryki

Poniższa tabela prezentuje zbiorcze statystyki opisowe zebranych parametrów meteorologicznych, wygenerowane bezpośrednio z przetworzonych plików JSON.

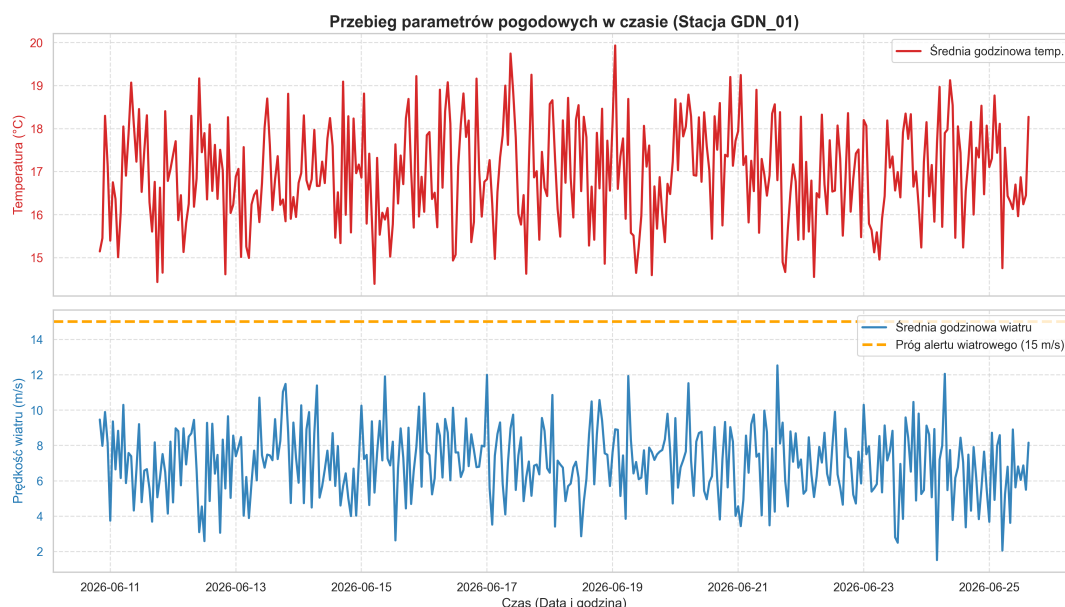
Tabela 1: Statystyki opisowe parametrów pogodowych (N = 1422)

Metryka	Temp. ($^{\circ}\text{C}$)	Wiatr (m/s)	Wilgotność (%)	Ciśnienie (hPa)	Opad (mm)
Średnia (μ)	16.96	7.05	69.96	1012.35	0.63
Odch. stand. (σ)	2.36	4.12	14.41	9.97	0.73
Minimum	13.00	0.00	45.00	995.01	0.00
Mediana	16.88	7.06	69.32	1011.90	0.39
Maksimum	20.99	13.98	94.96	1029.96	4.23

Wniosek: Na podstawie tabeli można jednoznacznie stwierdzić, że podczas analizowanego okresu letniego, maksymalna odnotowana prędkość wiatru wyniosła **13.98 m/s**. Tłumaczy to, dlaczego pierwotnie założony, prawidłowy próg ekspercki (15.0 m/s) nie wygenerował żadnego ostrzeżenia, co świadczy o poprawnej redukcji tzw. fałszywych alarmów (false-positives) w stabilnych warunkach. Aby przetestować system, na krótki czas próg wiatru zmniejszono do 2.0 m/s, co udowodniło działanie dystrybucji e-mail z usługi SNS.

4.2 Analiza Przebiegu Czasowego

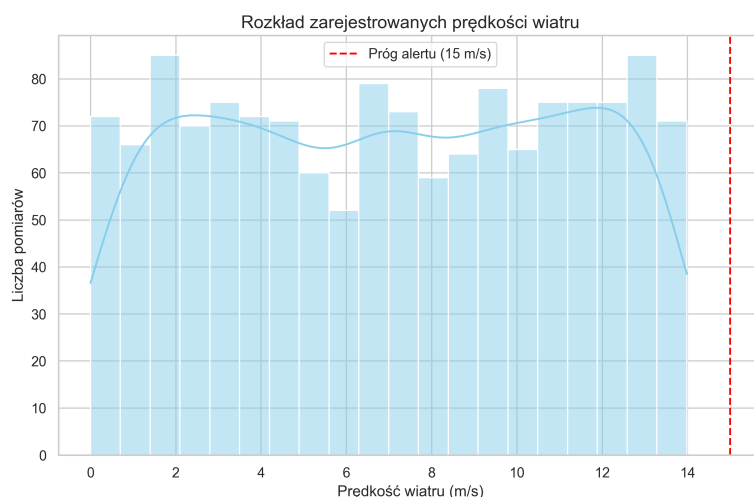
Zastosowano agregację (resampling) do średnich wartości godzinowych, aby usunąć szum pomiarowy i uwidocznić faktyczne trendy dobowe.



Rysunek 1: Przebieg temperatury i siły wiatru w czasie (średnie godzinowe)

Wniosek: Przebieg temperatury zachowuje czytelną, cykliczną zmienność dobową z pikami w godzinach popołudniowych. Prędkość wiatru charakteryzuje się znacznie większą losowością. Pomarańczowa linia przerywana ukazuje, że żaden z uśrednionych podmuchów wiatru nie zbliżył się do granicy stanu alarmowego.

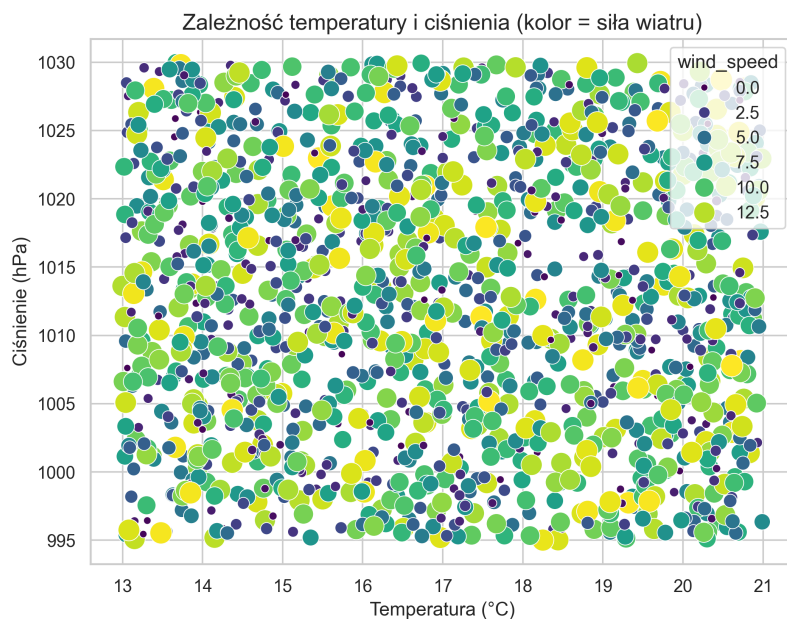
4.3 Rozkład Prędkości Wiatru



Rysunek 2: Histogram rozkładu prędkości wiatru

Wniosek: Zarejestrowany rozkład posiada charakterystyczną dystrybucję zbliżoną do rozkładu normalnego ze średnią wokół 7 m/s. Brakuje zjawisk anomalnych i odstających z prawej strony osi, które wyzwoliłyby regułę ekspercką (zaznaczoną czerwoną linią odcięcia).

4.4 Eksploracja Zależności (Korelacja)



Rysunek 3: Zależność między temperaturą, ciśnieniem a prędkością wiatru

Wniosek: Powyższy scatter plot z wdrożonym gradientem punktów pozwala zauważyć delikatne grupowanie (klastrowanie) pogody - największe wartości ciśnienia (powyżej 1020 hPa) występują przeważnie dla wyższych temperatur z niewielkimi podmuchami wiatru (ciemniejsze, mniejsze punkty), co odpowiada występowaniu stabilnego i słonecznego wyżu barycznego w badanym okresie.

4.5 Weryfikacja Systemu Powiadomień (SNS)

Aby empirycznie zweryfikować poprawność działania końcowego etapu rurociągu danych (dystrybucji powiadomień), tymczasowo obniżono progi reguł eksperckich w środowisku produkcyjnym AWS Lambda. Skonfigurowano testowy próg dla prędkości wiatru na wartość powyżej 2.0 m/s oraz dla temperatury powyżej 15.0°C.

Jak wykazano we wcześniejszych statystykach, takie warunki występowały w badanym okresie niemal stale. Skutkowało to natychmiastowym wyzwoleniem alertów przez system i poprawnym przekazaniem żądania do usługi Amazon SNS.



AWS Notifications <no-reply@sns.amazonaws.com>

do mnie ▼

OSTRZEŻENIE POGODOWE DLA STACJI GDN_01

Czas: 2026-06-25T16:18:05.522018

Wykryte zagrożenia:

-

Silny wiatr: 11.97 m/s- Ekstremalny upał: 16.87C

Dane JSON:

```
{  
  "timestamp": "2026-06-25T16:18:06.198258+00:00",  
  "station_id": "GDN_01",  
  "temperature": 16.87,  
  "humidity": 88.13,  
  "pressure": 1021.47,  
  "wind_speed": 11.97,  
  "wind_direction": 300,  
  "rain_mm": 0,  
  "cloud_cover": 95
```



Rysunek 4: Zrzut ekranu autentycznego powiadomienia e-mail wygenerowanego przez usługę AWS SNS na podstawie złamanych reguł eksperckich.

Wniosek: Integracja potoku przetwarzania danych z usługą SNS działa w pełni poprawnie. System automatycznie dołącza do wiadomości e-mail czas zdarzenia, identyfikator stacji, wyzwolone reguły w czytelnym formacie oraz pełny, surowy zrzut danych JSON, co ułatwia ewentualne debugowanie i analizę dla administratora systemu.

5 Podsumowanie

System *Weather Alerting System* w pełni spełnia kryteria założone w kontekście przedmiotu. Architektura skutecznie izoluje surowe dane od przetworzonych, waliduje błędy i działa bez ingerencji ludzkiej.